



Amdahl's Law vs. Gustafson's Law

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year II Semester

ACADEMIC YEAR : 2025 - 2026

Scalability & Speedup Performance

Speedup measures performance gains from executing code on multiple processors. Scalability evaluates how well speedup scales as both processor count and problem size increase.

Core Speedup Concepts

- **Speedup Formula (S_p):** $S_p = T_1 / T_p$, where T_1 is execution time on 1 processor, and T_p is execution time on p processors.
- **Ideal Speedup:** Linear speedup ($S_p = p$). Doubling processors should halve the execution time.
- **Real-World Limits:** Overhead (communication, scheduling, memory latency) and serial portions of code prevent ideal scaling.
- **Scalability Analysis:** Studies how speedup behaves as both processor count and workload size grow together.

Serial Bottlenecks

- **The Serial Fraction (s):** The portion of a program that cannot be parallelized (e.g. initialization, writing output files).
- **The Parallel Fraction (p):** The portion of code that can be executed concurrently across cores (e.g., matrix updates). Note: $s + p = 1$.
- **The Challenge:** No matter how many processors are added, the time to run the serial fraction ' s ' remains constant, creating a bottleneck.

Amdahl's Law (Fixed-Workload)

Gene Amdahl (1967) modeled speedup assuming a FIXED workload size:

Formula & Derivation

Given serial fraction s and parallel fraction p (with $s + p = 1$):

$$S_p = 1 / (s + p / N)$$

where N is the number of processors.

- **Limit as $N \rightarrow \infty$ (Asymptotic Speedup):**

$$S_\infty = 1 / s$$

If only 5% of your program is serial ($s = 0.05$), your MAXIMUM speedup is $1/0.05 = 20$, even with an infinite number of processors!

Implications of Amdahl's Law

- **Diminishing Returns:** Adding processors yields decreasing speedup gains because the serial fraction dominates execution time.
- **Critical Bottleneck:** Focus on optimizing the serial fraction to achieve higher speedup rather than simply scaling core counts.
- **Design Constraint:** Tells chip designers that adding hundreds of cores to a CPU is useless if code remains sequential.

Gustafson's Law (Scaled-Workload)

John Gustafson (1988) argued that problem sizes scale as computing power increases:

Formula & Derivation

Gustafson assumed parallel execution time is fixed, and problem size scales to match hardware size:

$$S_p = s + N \cdot (1 - s)$$

where N is the number of processors, and s is the serial fraction of the scaled workload.

- **Linear Scaling Concept:** Speedup increases linearly with processor count N . E.g., if $s = 0.05$ (5% serial), on 100 processors:

$S_{100} = 0.05 + 100 \cdot 0.95 = 95.05$. A very high speedup is achievable!

Implications of Gustafson's Law

- **Problem Size Scaling:** As computing power grows, we don't run the same small problem faster. Instead, we run larger, more complex simulations (e.g. higher-res weather models).
- **Linear Speedup:** If the workload scales, the serial fraction becomes negligible relative to the massive parallel processing volume, allowing linear speedups.
- **Optimistic View:** Proves that massively parallel supercomputer grids are highly effective for large-scale scientific modeling.



Comparison: Amdahl vs. Gustafson

A direct comparative summary of the two performance scaling laws:

Comparison Metric	Amdahl's Law	Gustafson's Law
Workload Model	Fixed Size (does not change)	Scaled Size (grows with hardware)
Speedup Behavior	Logarithmic curve; bounds speedup to 1/s	Linear growth line; no fixed limit
Hardware Focus	Optimize core latency (single-thread speed)	Maximize throughput (aggregate core counts)
Typical Application	Desktop apps, local tools, minor utilities	Supercomputers, AI training, weather models
Primary Goal	Minimize execution latency	Maximize problem accuracy & scope

Applications of Speedup Laws

Understanding speedup laws determines how companies scale hardware and software architectures:

Multicore CPU Design

Chip manufacturers balance single-thread clock speeds (Amdahl's bottleneck) with core counts, optimizing both sequential and parallel fractions.

Cloud Computing & Auto-scaling

Cloud platforms auto-scale nodes based on workload. If a task is parallelizable, Gustafson's law guarantees that adding instances scales throughput.

HPC Scientific Simulations

Climate models and astrophysics simulators scale grid nodes to increase grid resolution (Gustafson's law), achieving highly accurate models.

AI Training & LLM scaling

Training massive AI models requires thousands of GPUs. Gustafson's law justifies this scale, as training parameters are scaled to prevent bottlenecks.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management